LAPORAN PRAKTIKUM DEEP LEARNING

Attention dan Blok Transformer Sederhana

Cakrawala University

Kelompok 5

Identitas anggota kelompok

No.	Nama	NIM	Kontribusi
1	Tita Noviana	24120500011	25%
2	Titanio Yudista	24120500031	25%
3	Suci Fransisca Sisilia R	24120500008	25%
4	Fajar Dwiharjo	24130500010	25%

October 14, 2026

Contents

1 Pendahuluan **2**
- 1.1 Latar Belakang . . . 2
- 1.2 Tujuan Praktikum . . . 2
- 1.3 Sistematika Laporan . . . 2

2 Konfigurasi dan Lingkungan **2**
- 2.1 Artefak Reproduksi . . . 2

3 Dasar Implementasi **3**
- 3.1 Scaled Dot-Product Attention . . . 3
- 3.2 Multi-Head Attention . . . 3
- 3.3 Masking . . . 3
- 3.4 Positional Encoding . . . 3

4 Verifikasi Perhitungan Manual **4**
- 4.1 Data Contoh . . . 4
- 4.2 Perhitungan Tangan . . . 4
- 4.3 Perbandingan dengan Program . . . 4

5 Multi-Head Attention dan Jumlah Parameter **4**
- 5.1 Alur dan Shape . . . 4

6 Masking **5**
- 6.1 Padding Mask . . . 5
- 6.2 Causal Mask . . . 5

7 Positional Encoding **6**

8 Transformer Block **6**
- 8.1 Struktur Block . . . 6

9 Uji Fungsional **7**
- 9.1 Data dan Model . . . 7
- 9.2 Konfigurasi Pelatihan . . . 7
- 9.3 Hasil Pelatihan dan Overfit Satu Batch . . . 8

10 Verifikasi Implementasi **8**

11 Pertanyaan Analisis **8**
- 11.1 Shape Tensor . . . 9
- 11.2 Scaling pada Attention . . . 9
- 11.3 Padding Mask dan Causal Mask . . . 10
- 11.4 Positional Encoding . . . 10
- 11.5 Layer Normalization . . . 11
- 11.6 Kesalahan Implementasi . . . 12
  - 11.6.1 Deskripsi dan Mekanisme Masalah . . . 13
  - 11.6.2 Solusi Perbaikan dan Cara Verifikasi . . . 13
- 11.7 Kaitan dengan Model Recurrent . . . 14

11.8 Causal dan Bidirectional Attention 14
11.9 Pengayaan NON-RPS 15

12 Kesimpulan **15**

A Perintah Menjalankan Program **15**

B Kode atau Output Tambahan **15**

1 Pendahuluan

1.1 Latar Belakang

Attention memungkinkan setiap posisi dalam sekuens menghitung bobot terhadap posisi lain berdasarkan kesesuaian query dan key, kemudian menggabungkan value secara berbobot. Transformer memanfaatkan mekanisme ini tanpa pemrosesan sekuens berulang, sehingga hubungan antarposisi dapat diperiksa melalui matriks attention.

1.2 Tujuan Praktikum

Praktikum ini bertujuan mengimplementasikan scaled dot-product attention, multi-head attention, masking, positional encoding sinusoidal, dan blok transformer sederhana. Implementasi diverifikasi melalui perhitungan manual, pemeriksaan shape, jumlah parameter, bobot attention, serta uji fungsional pada data sintetis.

1.3 Sistematika Laporan

Laporan ini menyajikan implementasi teknis lengkap dari seluruh bagian Instruksi praktikum serta pembahasan analitis mendalam untuk Pertanyaan Analisis nomor 1 hingga 9, yang didukung oleh verifikasi komputasi numerik dan bukti empiris.

2 Konfigurasi dan Lingkungan

Table 1: Konfigurasi eksperimen

Parameter	Nilai
Framework	PyTorch
Seed	10
Format tensor	`(B,T,d_model)`
Batch size, B	2
Panjang sekuens, T	4
Dimensi model, d_{model}	8
Jumlah head, h	2
Dimensi per head, d_{head}	4
Dimensi feed-forward, d_{ff}	32
Aktivasi FFN	GELU
Optimizer	Adam
Learning rate	0.02

2.1 Artefak Reproduksi

Source code utama tersedia pada berkas `attention_transformer.py`. Hasil verifikasi disimpan dalam `artifacts/verification_report.json`, ringkasan

pada `artifacts/verification_report.md`, serta metrik pelatihan dalam `artifacts/training_metrics.csv`.

3 Dasar Implementasi

3.1 Scaled Dot-Product Attention

Scaled dot-product attention dihitung menggunakan:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V. \tag{1}$$

Pada implementasi, skor dihitung dengan perkalian QK^T, dibagi dengan $\sqrt{d_k}$, lalu diproses oleh softmax pada dimensi key. Jika mask diberikan, posisi terlarang diisi dengan nilai sangat negatif sebelum softmax. Fungsi mengembalikan output, skor, dan bobot attention agar setiap tahap dapat diverifikasi.

3.2 Multi-Head Attention

Input diproyeksikan secara terpisah menjadi Q, K, dan V dengan dimensi akhir 8. Masing-masing tensor diubah dari $(B, T, 8)$ menjadi $(B, h, T, d_{\text{head}}) = (B, 2, T, 4)$. Attention dihitung secara paralel pada dua head, hasilnya digabung kembali menjadi $(B, T, 8)$, kemudian melewati output projection.

3.3 Masking

Padding mask diterapkan pada dimensi key untuk menghilangkan pengaruh token padding. Pada contoh batch, posisi padding adalah posisi keempat pada sampel pertama serta posisi ketiga dan keempat pada sampel kedua. Causal mask berupa matriks segitiga atas yang melarang setiap query melihat posisi masa depan. Mask diterapkan sebelum softmax sehingga bobot posisi terlarang menjadi nol. Implementasi juga menolak baris yang seluruh key-nya dilarang.

3.4 Positional Encoding

Positional encoding sinusoidal ditambahkan ke embedding input menggunakan

$$PE_{(p,2i)} = \sin\left(\frac{p}{10000^{2i/d_{\text{model}}}}\right), \tag{2}$$

$$PE_{(p,2i+1)} = \cos\left(\frac{p}{10000^{2i/d_{\text{model}}}}\right). \tag{3}$$

Implementasi menghasilkan encoding berukuran $(4, 8)$ dan menambahkannya secara broadcasting ke input berukuran $(2, 4, 8)$.

4 Verifikasi Perhitungan Manual

4.1 Data Contoh

Gunakan data wajib dari instruksi:

$$q = [1, 0, 1, 0], \quad k_1 = [1, 0, 0, 0], \quad k_2 = [0, 1, 0, 0],$$
$$v_1 = [1, 0], \quad v_2 = [0, 1].$$

4.2 Perhitungan Tangan

$$\begin{aligned} qK^T &= [1, 0], \\ \frac{qK^T}{\sqrt{d_k}} &= [0.5, 0], \\ \text{softmax}([0.5, 0]) &= [0.622459,\ 0.377541], \\ \text{Attention}(q, K, V) &= [0.622459,\ 0.377541]. \end{aligned}$$

4.3 Perbandingan dengan Program

Table 2: Perbandingan hasil manual dan program

Komponen	Hasil manual	Hasil program
Skor terskala	$[0.5, 0]$	$[0.5, 0]$
Bobot attention	$[0.622459, 0.377541]$	$[0.622459, 0.377541]$
Output	$[0.622459, 0.377541]$	$[0.622459, 0.377541]$
Error maksimum absolut	0.0	
Status	PASS	

5 Multi-Head Attention dan Jumlah Parameter

5.1 Alur dan Shape

Alur implementasi adalah proyeksi linear Q/K/V, split head, scaled dot-product attention, transpose dan concat antar-head, kemudian output projection. Untuk konfigurasi eksperimen, setiap proyeksi linear berukuran 8×8 dengan 8 bias.

Table 3: Jumlah parameter multi-head attention

Komponen	Bobot	Bias	Total
Proyeksi Q	$8 \times 8 = 64$	8	72
Proyeksi K	$8 \times 8 = 64$	8	72
Proyeksi V	$8 \times 8 = 64$	8	72
Output projection	$8 \times 8 = 64$	8	72
Total manual	256	32	**288**
Total program	**288**		**PASS**

6 Masking

6.1 Padding Mask

Mask padding yang digunakan adalah

$$\begin{bmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 1 \end{bmatrix},$$

dengan 1 menunjukkan key yang dilarang. Hasil program menunjukkan bobot terbesar pada seluruh posisi terlarang adalah 0.0. Jumlah setiap baris attention berada dalam toleransi 1.2×10^{-7} dari satu.

Table 4: Bukti padding mask

Pemeriksaan	Hasil
Posisi padding yang dilarang	Sampel 1: k_4; sampel 2: k_3, k_4
Bobot maksimum posisi terlarang	0.0
Deviasi maksimum jumlah baris dari 1	1.2×10^{-7}
Status	PASS

6.2 Causal Mask

Mask causal yang digunakan adalah

$$\begin{bmatrix} 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix}.$$

Dengan demikian, query pada posisi t hanya dapat memperhatikan key pada posisi hingga t. Hasil program menunjukkan bobot maksimum pada posisi masa depan adalah 0.0 dan setiap baris valid tetap ternormalisasi.

Table 5: Bukti causal mask

Pemeriksaan	Hasil
Posisi masa depan yang dilarang	Segitiga atas
Bobot maksimum posisi terlarang	0.0
Deviasi maksimum jumlah baris dari 1	1.2×10^{-7}
Baris tanpa key valid ditolak	Ya
Status	PASS

7 Positional Encoding

Table 6: Verifikasi positional encoding

Pemeriksaan	Hasil
Shape encoding	$(4, 8)$
Shape setelah broadcasting	$(2, 4, 8)$
Posisi berbeda	Ya
Status	PASS

Nilai dua posisi pertama yang dihasilkan program adalah

$PE_0 = [0, 1, 0, 1, 0, 1, 0, 1]$,
$PE_1 = [0.841471, 0.540302, 0.099833, 0.995004, 0.0099998, 0.999950, 0.0010000, 1.000000]$.

Keduanya berbeda, sehingga pemeriksaan runtime untuk keberbedaan posisi dinyatakan lulus.

8 Transformer Block

8.1 Struktur Block

Block transformer terdiri atas multi-head attention, residual connection, layer normalization, feed-forward network, residual connection kedua, dan layer normalization kedua. Dalam kode, urutannya adalah

$$X \longrightarrow \mathrm{MHA}(X) \longrightarrow \mathrm{LayerNorm}(X + \mathrm{MHA}(X)) \longrightarrow \mathrm{FFN} \longrightarrow \mathrm{LayerNorm}(\cdot).$$

FFN menggunakan layer linear $8 \to 32$, aktivasi GELU, dan layer linear $32 \to 8$.

Table 7: Shape tensor pada transformer block

Tensor/tahap	Shape
Input embedding	$(2, 4, 8)$
Q, K, V sebelum split head	$(2, 4, 8)$
Q, K, V sesudah split head	$(2, 2, 4, 4)$
Matriks skor per head	$(2, 2, 4, 4)$
Bobot attention per head	$(2, 2, 4, 4)$
Keluaran head sebelum concat	$(2, 2, 4, 4)$
Setelah concat	$(2, 4, 8)$
Keluaran MHA	$(2, 4, 8)$
Keluaran FFN	$(2, 4, 8)$
Keluaran akhir block	$(2, 4, 8)$

Table 8: Jumlah parameter transformer block

Komponen	Jumlah parameter
Multi-head attention	288
Linear FFN pertama	288
Linear FFN kedua	264
Dua layer normalization	32
Total manual	**872**
Total program	**872**

9 Uji Fungsional

9.1 Data dan Model

Uji fungsional menggunakan klasifikasi sekuens pendek menjadi dua kelas pada data sintetis. Label dibentuk secara deterministik dari tanda rata-rata fitur pertama sepanjang sekuens. Data train terdiri atas 32 sekuens dan data test terdiri atas 64 sekuens. Setiap sekuens memiliki panjang 4 dan dimensi fitur 8. Model menambahkan positional encoding, memproses input dengan satu transformer block, melakukan mean pooling pada dimensi sekuens, lalu menghasilkan dua logit kelas.

9.2 Konfigurasi Pelatihan

Model dilatih selama 100 epoch menggunakan Adam dengan learning rate 0.02 dan cross-entropy loss. Seed acak ditetapkan ke 10. Data train diproses sebagai satu batch penuh pada setiap epoch sehingga penurunan loss dan kenaikan akurasi dapat dipakai sebagai bukti kemampuan overfit pada batch kecil.

9.3 Hasil Pelatihan dan Overfit Satu Batch

Table 9: Ringkasan hasil uji fungsional

Metrik	Nilai
Loss awal	0.734517
Loss akhir	0.000168
Akurasi train akhir	1.0000
Akurasi test	0.8750
Overfit satu batch	PASS

Metrik per epoch tersedia pada `artifacts/training_metrics.csv`. Kurva loss dan akurasi dapat dibuat dari file tersebut jika visualisasi diperlukan.

10 Verifikasi Implementasi

Table 10: Ringkasan pemeriksaan implementasi

Pemeriksaan	Status	Bukti
Perhitungan manual attention	PASS	Error maksimum 0.0
Jumlah parameter MHA	PASS	288 = 288
Jumlah parameter block	PASS	872 = 872
Shape tensor	PASS	Assertion runtime
Padding mask	PASS	Bobot terlarang 0.0
Causal mask	PASS	Bobot masa depan 0.0
Positional encoding	PASS	Shape dan posisi berbeda
Overfit satu batch	PASS	Akurasi train 1.0000

Seluruh pemeriksaan dilakukan dengan assertion runtime pada skrip `attention_transformer.py`. Ringkasan numerik lengkap dapat dilihat pada berkas `artifacts/verification_report.json`; ringkasannya tersedia pada dokumen `artifacts/verification_report.md`.

11 Pertanyaan Analisis

Bagian ini menyajikan analisis mendalam terhadap konsep teoretis dan teknis di balik implementasi mekanisme attention dan blok transformer, mencakup analisis dimensi tensor, faktor penskalaan skor, mekanisme penyembunyian (*masking*), representasi posisi, teknik normalisasi, penanganan galat implementasi, serta perspektif evolusi arsitektur dan inovasi modern.

11.1 Shape Tensor

Untuk konfigurasi $d_model = 8$, $h = 2$ (sehingga $d_head = d_model/h = 4$), $T = 4$, dan $B = 2$, shape tensor pada setiap tahap `MultiHeadAttention` adalah sebagai berikut. Nilai ini sesuai dengan hasil pemeriksaan program pada `shape_and_parameter_verification` di `attention_transformer.py` dan Tabel 10.

Table 11: Shape tensor pada setiap tahap multi-head attention

Tensor	Shape	Keterangan
Input	$(B, T, d_{\text{model}}) = (2, 4, 8)$	Masukan blok transformer
Q, K, V (sebelum split)	$(2, 4, 8)$	Keluaran proyeksi linear q, k, v
Q, K, V (setelah split)	$(B, h, T, d_{\text{head}}) = (2, 2, 4, 4)$	Setelah `view` + `transpose(1,2)`
Matriks skor $QK^\top/\sqrt{d_k}$	$(B, h, T, T) = (2, 2, 4, 4)$	Skor sebelum softmax, per head
Bobot attention	$(2, 2, 4, 4)$	Normalisasi softmax ($\sum = 1$)
Output per head	$(2, 2, 4, 4)$	weights $\times V$, sebelum concat
Output setelah concat	$(B, T, d_{\text{model}}) = (2, 4, 8)$	`transpose` + `view` kembali ke d_{model}
Keluaran akhir MHA	$(2, 4, 8)$	Setelah `output_projection`

Dimensi batch B dan panjang sekuens T dipertahankan di seluruh tahap, sedangkan dimensi fitur berpindah dari $d_model = 8$ menjadi $h \times d_head = 2 \times 4$ saat kepala attention dipisah, lalu kembali menjadi $d_model = 8$ setelah hasil setiap kepala digabungkan (concat) dan melewati `output_projection`. Matriks skor dan bobot attention berbentuk (B, h, T, T) karena setiap query dari tiap kepala dibandingkan dengan seluruh T key pada kepala yang sama.

11.2 Scaling pada Attention

Skor attention $QK^\top$ dibagi dengan $\sqrt{d_k}$ (pada implementasi ini $\sqrt{d_head} = \sqrt{4} = 2$) karena, jika elemen q dan k diasumsikan memiliki rata-rata 0 dan varians 1 serta saling bebas, hasil dot product $q \cdot k = \sum_{i=1}^{d_k} q_i k_i$ memiliki varians yang tumbuh linear terhadap d_k. Tanpa pembagian ini, magnitudo skor akan semakin besar seiring bertambahnya d_k, sehingga skor antar key bisa memiliki selisih yang sangat besar. Pembagian dengan $\sqrt{d_k}$ menormalkan varians skor agar tetap mendekati 1 berapa pun nilai d_k, sehingga skor berada pada rentang yang stabil sebelum masuk ke softmax.

Jika pembagian tersebut dihilangkan, skor mentah $QK^\top$ cenderung memiliki magnitudo besar (terutama untuk d_k yang besar). Softmax dari skor bermagnitudo besar akan *saturasi*: distribusi bobot menjadi sangat tajam dan mendekati one-hot pada key dengan skor tertinggi, sementara bobot pada key lain mendekati nol. Pada daerah saturasi ini, turunan fungsi softmax mendekati nol, sehingga gradien yang mengalir balik ke Q dan K saat training menjadi sangat kecil (vanishing gradient). Akibatnya proses optimisasi melambat atau bahkan berhenti belajar, dan perilaku ini akan semakin parah ketika d_model atau d_head diperbesar.

11.3 Padding Mask dan Causal Mask

Kedua jenis mask pada `attention_transformer.py` bekerja dengan mekanisme yang sama, yaitu memberi nilai sangat negatif (`torch.finfo(scores.dtype).min`) pada skor di posisi terlarang sebelum softmax dijalankan (fungsi `scaled_dot_product_attention`), sehingga bobot attention pada posisi tersebut menjadi 0 setelah softmax. Perbedaan keduanya terletak pada *posisi mana yang dilarang* dan *alasan pelarangannya*:

- **Padding mask** melarang attention membaca token padding (token isian yang ditambahkan agar panjang sekuens dalam satu batch sama) pada dimensi *key*. Mask ini sama untuk seluruh posisi query dalam satu sekuens — pada implementasi ini direpresentasikan sebagai `padding_mask[:, None, None, :]` berbentuk $(B, 1, 1, T)$ yang di-broadcast ke seluruh head dan seluruh baris query. Tujuannya adalah mencegah token padding yang tidak memiliki makna ikut memengaruhi hasil representasi token lain.
- **Causal mask** melarang query pada posisi t membaca key pada posisi $t' > t$ (masa depan), terlepas dari apakah token tersebut padding atau bukan. Mask ini dibentuk dengan matriks segitiga atas `torch.triu(..., diagonal=1)` berbentuk (T, T) yang sama untuk seluruh batch dan head. Tujuannya adalah menjaga sifat autoregresif, yaitu prediksi pada posisi t hanya boleh bergantung pada token $1, \ldots, t$.

Bukti bahwa kedua mask bekerja dengan benar ditunjukkan oleh `masking_verification`: bobot attention maksimum pada posisi terlarang untuk padding mask dan causal mask sama-sama 0.0 (lihat Tabel 10), sementara jumlah bobot pada setiap baris yang valid tetap 1.0.

Jika causal mask tidak diterapkan pada model autoregresif, query pada posisi t dapat melihat key pada posisi $t' > t$, yaitu token yang seharusnya belum "diketahui" model pada saat prediksi dibuat. Saat training, model dapat "mencontek" dengan memanfaatkan informasi token masa depan untuk memprediksi token saat ini, sehingga loss training terlihat rendah namun model tidak benar-benar mempelajari distribusi bersyarat $P(x_t \mid x_{<t})$ yang sesungguhnya. Pada saat inferensi, token masa depan belum tersedia (dihasilkan satu per satu), sehingga terjadi ketidaksesuaian antara kondisi training dan inferensi (train–inference mismatch) yang membuat kualitas prediksi menurun drastis dibandingkan hasil pada data training.

11.4 Positional Encoding

Positional encoding mutlak diperlukan karena operasi inti self-attention pada Transformer bersifat *permutation-equivariant* (atau *order-invariant* terhadap himpunan token). Komputasi dot-product $QK^\top$ dan perkalian bobot attention dengan V hanya mengevaluasi kesamaan kandungan fitur antarvektor representasi tanpa memperhitungkan urutan kemunculan posisi masing-masing token dalam sekuens. Berbeda dengan Recurrent Neural Network (RNN) yang memproses data langkah demi langkah secara sekuensial atau Convolutional Neural Network (CNN) yang memiliki bias induktif lokal berdasarkan ukuran kernel spasial, Transformer memproses seluruh posisi token secara serentak (paralel). Tanpa penambahan sinyal posisi, susunan kata yang berbeda seperti "kucing mengejar tikus" dan "tikus mengejar kucing" akan menghasilkan kumpulan vektor representasi keluaran yang bernilai identik (hanya letak barisnya yang

bertukar). Oleh karena itu, informasi letak absolut atau relatif harus diinjeksikan secara eksplisit ke dalam embedding masukan.

Pendekatan **sinusoidal positional encoding** (Vaswani et al., 2017) dan **learned position embedding** (sebagaimana digunakan pada BERT dan GPT-2) memiliki karakteristik dan perbandingan mendasar sebagai berikut:

1. **Jumlah Parameter**: Pendekatan sinusoidal dihitung secara deterministik menggunakan kombinasi fungsi trigonometri sinus dan kosinus pada frekuensi geometris yang telah ditentukan sebelumnya ($PE_{(pos,2i)} = \sin(pos/10000^{2i/d_{\text{model}}})$ dan $PE_{(pos,2i+1)} = \cos(pos/10000^{2i/d_{\text{model}}})$), sehingga membutuhkan 0 parameter yang dapat dilatih (*zero trainable parameters*). Sebaliknya, learned position embedding mengalokasikan tabel matriks bobot yang dapat dilatih (`nn.Embedding(max_length, d_model)`), sehingga menambah parameter sebanyak max_length $\times\, d_{\text{model}}$ yang harus dioptimasi lewat gradien balik.

2. **Sifat Pergeseran Relatif**: Pendekatan sinusoidal memiliki keunggulan matematis berkat identitas trigonometri $\sin(\alpha+\beta) = \sin\alpha\cos\beta+\cos\alpha\sin\beta$. Untuk sembarang jarak pergeseran tetap k, vektor PE_{pos+k} dapat diproyeksikan secara linear dari PE_{pos} melalui transformasi rotasi linear berdimensi 2×2 pada setiap pasangan kanal frekuensi. Karakteristik ini memudahkan perhatian model mengenali jarak spasial relatif antarposisi. Sementara itu, pada learned embedding, hubungan jarak relatif antarposisi harus dipelajari secara implisit dari variasi data latih.

3. **Kemampuan Ekstrapolasi Sekuens**: Rumus sinusoidal terdefinisi untuk sembarang nilai posisi bulat non-negatif $pos \geq 0$, sehingga secara teoretis dapat langsung dievaluasi pada sekuens yang lebih panjang daripada panjang maksimum sekuens saat pelatihan tanpa mengubah struktur model. Sebaliknya, learned position embedding dibatasi secara ketat oleh batas max_length yang dialokasikan saat inisialisasi; posisi di luar batas tersebut tidak memiliki vektor bobot dan tidak dapat diproses saat inferensi.

Pada implementasi `attention_transformer.py`, kelas `SinusoidalPositionalEncoding` disimpan sebagai buffer (`register_buffer`) berukuran $(1, 128, 8)$ tanpa menambah parameter pada blok transformer, dan terbukti menghasilkan vektor posisi unik pada setiap indeks sekuens (`assert not torch.allclose(positions[0], positions[1])`).

11.5 Layer Normalization

Layer Normalization (LN) secara fundamental jauh lebih sesuai daripada Batch Normalization (BN) pada arsitektur Transformer untuk pemrosesan data sekuens karena perbedaan arah komputasi statistik normalisasinya:

1. **Dimensi Komputasi Statistik**:

   - Batch Normalization menghitung rata-rata dan varians sepanjang dimensi *batch* B (serta dimensi sekuens T) secara terpisah untuk setiap kanal fitur j:

$$\mu_{BN,j} = \frac{1}{B \cdot T} \sum_{b=1}^{B} \sum_{t=1}^{T} x_{b,t,j}, \quad \sigma^2_{BN,j} = \frac{1}{B \cdot T} \sum_{b=1}^{B} \sum_{t=1}^{T} (x_{b,t,j} - \mu_{BN,j})^2. \tag{4}$$

Artinya, normalisasi satu sampel bergantung secara erat pada nilai sampel-sampel lain yang berada dalam satu batch.

- Layer Normalization menghitung rata-rata dan varians sepanjang dimensi *fitur/hidden* (d_{model}) secara independen untuk setiap token individual pada posisi waktu t dalam sampel b:

$$\mu_{LN,b,t} = \frac{1}{d_{\text{model}}} \sum_{j=1}^{d_{\text{model}}} x_{b,t,j}, \quad \sigma^2_{LN,b,t} = \frac{1}{d_{\text{model}}} \sum_{j=1}^{d_{\text{model}}} (x_{b,t,j} - \mu_{LN,b,t})^2. \tag{5}$$

2. **Penanganan Panjang Sekuens Dinamis dan Padding Mask**: Pada pemrosesan sekuens teks nyata, panjang kalimat bervariasi sehingga token *padding* ditambahkan untuk menyamakan ukuran tensor dalam satu batch. Jika BN diterapkan, kehadiran token padding atau sebaran panjang token valid yang tidak merata akan mencemari nilai rata-rata dan varians batch, merusak representasi token kata yang sebenarnya. Sebaliknya, LN beroperasi secara tertutup pada masing-masing token tanpa saling silang antarposisi waktu atau antarsampel, sehingga kebal terhadap keberadaan padding pada sampel lain.

3. **Invariansi terhadap Ukuran Batch (Batch Size Sensitivity)**: Komputasi perhatian pada Transformer memiliki kompleksitas memori kuadratik $O(T^2)$, sehingga pada saat melatih model transformer besar, ukuran batch per GPU sering kali harus disetel sangat kecil (seperti $B = 2$ pada eksperimen praktikum ini). Pada ukuran batch yang kecil, estimasi rata-rata dan varians BN menjadi sangat bising (*noisy*) dan mengalami fluktuasi ekstrem yang merusak konvergensi optimasi. Sementara itu, kestabilan estimasi statistik LN ditentukan oleh besarnya dimensi fitur d_{model}, bukan ukuran batch B, sehingga LN tetap bekerja dengan stabil bahkan pada batch berukuran 1 (*single-sample learning*).

4. **Kesesuaian dengan Inferensi Autoregresif**: BN memerlukan pencatatan statistik rata-rata dan varians bergerak (*running mean* dan *running variance*) selama pelatihan untuk digunakan saat tahap inferensi. Namun, pada proses inferensi autoregresif (menghasilkan token kata satu per satu secara berulang), distribusi masukan sekuens yang bertambah secara dinamis sering kali berbeda dengan distribusi sekuens penuh saat pelatihan (*distributional mismatch*). LN tidak memerlukan penyimpanan statistik berjalan sama sekali karena selalu menghitung normalisasi secara langsung (*on-the-fly*) dari representasi vektor aktivasi token yang sedang diproses.

Pada implementasi `TransformerBlock`, penggunaan `nn.LayerNorm(d_model)` pada sub-lapisan perhatian dan sub-lapisan feed-forward memastikan gradien mengalir secara stabil pada batch kecil ($B = 2$), terbukti dari keberhasilan proses pelatihan yang mampu menurunkan loss dari 0.734517 ke 0.000168.

11.6 Kesalahan Implementasi

Salah satu kesalahan implementasi yang sangat umum namun fatal dalam pengembangan blok Multi-Head Attention adalah **ketiadaan pemanggilan `.contiguous()` setelah operasi manipulasi permutasi dimensi (`.transpose(1, 2)`) sebelum memanggil `.view()`** pada tahap penggabungan keluaran multi-head.

11.6.1 Deskripsi dan Mekanisme Masalah

Pada modul `MultiHeadAttention`, setelah fungsi `scaled_dot_product_attention` selesai dieksekusi, tensor keluaran tiap kepala (`output_heads`) memiliki bentuk tensor $(B, h, T, d_{\text{head}})$. Agar dapat diproyeksikan kembali melalui `output_projection` berukuran $(d_{\text{model}} \rightarrow d_{\text{model}})$, dimensi head (h) dan panjang sekuens (T) harus ditukar kembali ke urutan semula, lalu digabungkan menjadi (B, T, d_{model}). Jika penggabungan ditulis secara langsung sebagai berikut:

```
# Potensi kesalahan implementasi:
concatenated = output_heads.transpose(1, 2).view(
    query.shape[0], query.shape[1], self.d_model
)
```

PyTorch akan langsung melempar galat runtime:

```
RuntimeError: view size is not compatible with input tensor's
size and stride (at least one dimension spans across two
contiguous subspaces). Use .contiguous() to view() or consider
reshape().
```

Penyebab teknis dari galat ini adalah bahwa operasi `transpose()` pada PyTorch tidak menyusun ulang atau menyalin elemen tensor di memori fisik (RAM/VRAM), melainkan hanya mengubah skema langkah memori (*strides*). Akibatnya, tata letak memori fisik tensor tidak lagi berurutan secara linier (*non-contiguous*). Karena metode `.view()` mensyaratkan memori yang utuh dan berurutan agar dapat memetakan dimensi baru secara langsung tanpa alokasi memori ganda, pemanggilan `.view()` pada tensor hasil transposisi akan gagal.

11.6.2 Solusi Perbaikan dan Cara Verifikasi

Perbaikan dilakukan dengan menambahkan pemanggilan eksplisit `.contiguous()` tepat sebelum metode `.view()`, sebagaimana diterapkan pada baris 108–110 `attention_transformer.py`:

```
# Implementasi yang benar:
concatenated = output_heads.transpose(1, 2).contiguous().view(
    query.shape[0], query.shape[1], self.d_model
)
```

Pemanggilan `.contiguous()` memaksa PyTorch mengalokasikan blok memori fisik yang tertata secara linier berurutan sesuai dengan urutan dimensi baru hasil transposisi, sehingga operasi `.view()` dapat dijalankan secara valid dan efisien.

Perbaikan ini diverifikasi melalui rangkaian pengujian pada skrip:

1. **Verifikasi Shape Runtime**: Pada fungsi `shape_and_parameter_verification`, variabel `concatenated` dipastikan menghasilkan shape yang presisi $(B, T, d_{\text{model}}) = (2, 4, 8)$ dan keluaran proyeksi akhir `mha_output` berukuran $(2, 4, 8)$ tanpa runtime exception (`assert actual_shapes == expected_shapes`).

2. **Verifikasi Aliran Gradien (Backpropagation)**: Keutuhan penyusunan memori diuji lebih lanjut pada pengujian fungsional `train_functional_model()`. Penggabungan kepala attention dieksekusi bolak-balik selama 100 epoch pelatihan

(forward dan backward pass). Gradien terbukti mengalir tanpa kendala ke seluruh proyeksi matriks (W_Q, W_K, W_V, W_O), dibuktikan dengan tercapainya akurasi pelatihan sempurna 1.0000 dan kelulusan pengujian overfit satu batch (`PASS`).

11.7 Kaitan dengan Model Recurrent

Model recurrent seperti RNN dan LSTM memproses sekuens secara berurutan: representasi posisi ke-t baru dapat dihitung setelah posisi $t-1$ selesai. Akibatnya, komputasi tidak dapat diparalelkan sepanjang dimensi waktu, dan informasi dari token yang berjarak jauh harus melewati banyak langkah rekursif sehingga mudah melemah (masalah ketergantungan jarak jauh). Pada arsitektur seq2seq klasik, seluruh sekuens masukan bahkan dipadatkan menjadi satu vektor konteks, yang menjadi bottleneck informasi.

Attention menyelesaikan masalah tersebut dengan memungkinkan setiap posisi langsung mengakses seluruh posisi lain dalam satu langkah, sehingga jalur informasi antarposisi berjarak konstan dan tidak bergantung pada rekursi. Pada modul yang diimplementasikan, komponen yang menjawab masalah ini adalah fungsi `scaled_dot_product_attention`. Matriks skor berukuran (B, h, T, T) dihitung sekaligus melalui perkalian $QK^\top$, sehingga seluruh pasangan query-key dievaluasi secara paralel tanpa loop atas dimensi waktu. Metode `MultiHeadAttention.forward` juga memproses keempat posisi dalam satu operasi tensor. Matriks bobot yang tersimpan pada atribut `last_attention_weights` memperlihatkan bahwa setiap query dapat menerima kontribusi dari setiap key yang tidak dilarang mask, yaitu koneksi langsung antarposisi tanpa melewati state tersembunyi.

11.8 Causal dan Bidirectional Attention

Perbedaan utama terletak pada siapa yang boleh dilihat oleh setiap token. Pada keluarga decoder-only seperti GPT, causal mask dipakai agar representasi posisi t hanya bergantung pada $x_{\leq t}$. Training tetap dapat diparalelkan melalui teacher forcing, tetapi setiap posisi tidak pernah melihat konteks di kanannya, sehingga konsisten dengan proses generasi token demi token saat inferensi. Pada keluarga encoder-only seperti BERT, attention bersifat dua arah: setiap token melihat seluruh sekuens. Ini cocok untuk tugas pemahaman seperti klasifikasi, tetapi model tidak dapat langsung dipakai untuk generasi autoregresif. Keluarga encoder-decoder seperti Transformer 2017 dan T5 menggabungkan keduanya: encoder bersifat dua arah, sedangkan decoder menggunakan causal mask dan cross-attention ke keluaran encoder.

Dalam repository ini, `SequenceClassifier` memanggil `self.block(encoded)` tanpa mask, sehingga attention-nya bersifat dua arah. Pilihan ini tepat karena tugasnya adalah klasifikasi seluruh sekuens dengan mean pooling, bukan prediksi token berikutnya. Jika model yang sama diubah menjadi language model, causal mask perlu diberikan secara eksplisit. Pemeriksaan `masking_verification` membuktikan mekanisme causal mask berfungsi, tetapi mask tersebut tidak digunakan pada model klasifikasi.

Dua komponen yang tetap bertahan sejak Transformer 2017 adalah (1) multi-head scaled dot-product attention sebagai operator utama untuk pencampuran informasi antarposisi, dan (2) residual connection yang dikombinasikan dengan layer normalization pada setiap sub-lapisan. Keduanya masih dipakai pada hampir seluruh model modern. Detail lain telah banyak berubah, misalnya posisi layer normalization (dari post-LN

menjadi pre-LN), aktivasi FFN (dari ReLU menjadi GELU atau SwiGLU), dan cara encoding posisi.

11.9 Pengayaan NON-RPS

Varian yang dipilih adalah rotary position embedding (RoPE). Pada implementasi, positional encoding bersifat absolut dan aditif: vektor sinusoidal langsung dijumlahkan ke embedding input sebelum blok transformer, sehingga informasi posisi bercampur dengan informasi isi token dan hanya tersedia sebagai nilai absolut. RoPE tidak menambahkan apa pun ke embedding. Sebagai gantinya, setiap pasangan dimensi pada query dan key diputar dengan sudut yang proporsional terhadap posisinya, sehingga hasil dot product $q_m^\top k_n$ hanya bergantung pada selisih relatif $m - n$.

Perbaikannya ada tiga. Pertama, informasi posisi relatif langsung masuk ke skor attention, tempat informasi tersebut paling berguna. Kedua, representasi isi token tidak terganggu oleh vektor posisi. Ketiga, dengan teknik penskalaan frekuensi, RoPE umumnya lebih baik dalam mengekstrapolasi ke sekuens yang lebih panjang daripada sinusoidal absolut. Menariknya, frekuensi $\theta_i = 10000^{-2i/d}$ yang digunakan RoPE sama dengan jadwal frekuensi pada `SinusoidalPositionalEncoding`, sehingga keduanya berbagi dasar matematika yang sama dan perbedaannya ada pada cara penerapannya. Penerapan RoPE pada kode ini cukup dilakukan setelah `_split_heads` dan sebelum `scaled_dot_product_attention`, hanya pada Q dan K, dengan syarat $d_{\text{head}} = 4$ genap sehingga pasangan dimensi terbentuk. RoPE tidak menambah parameter, sehingga jumlah 288 parameter multi-head attention dan 872 parameter block tetap berlaku.

12 Kesimpulan

Bagian Instruksi berhasil diimplementasikan dan diverifikasi. Perhitungan manual scaled dot-product attention menghasilkan error maksimum absolut 0.0. Multi-head attention memiliki 288 parameter dan transformer block memiliki 872 parameter, sesuai dengan perhitungan manual. Padding mask dan causal mask menghasilkan bobot nol pada seluruh posisi terlarang. Positional encoding menghasilkan shape yang sesuai, yaitu $(4, 8)$ sebelum broadcasting dan $(2, 4, 8)$ setelah ditambahkan ke input. Pada uji fungsional, loss turun dari 0.734517 menjadi 0.000168 dan akurasi train mencapai 1.0000, sehingga kriteria overfit satu batch terpenuhi. Akurasi test yang diperoleh adalah 0.8750 pada data sintetis yang digunakan.

A Perintah Menjalankan Program

```
python attention_transformer.py
```

B Kode atau Output Tambahan

Source code lengkap tersedia pada `attention_transformer.py`. Output verifikasi yang digunakan untuk mengisi laporan ini tersedia pada `artifacts/verification_report.json`, sedangkan metrik per epoch tersedia pada `artifacts/training_metrics.csv`.